

⚡ Performance notes: how this app stays fast (and how experts speed up Apps Script)

The #1 rule of Google Apps Script performance: **the slow part is the calls to Google's services, not the JavaScript**. Every `getValue()`, `setValue()`, `getRange()`, `getSheetByName()`, `MailApp.sendEmail()`, and every `google.script.run` call crosses a network/service boundary that costs tens to hundreds of milliseconds. JS loops and math are essentially free by comparison. So the whole game is: **make fewer service calls, and move more data per call**.

Below are the techniques the pros use, and exactly where each is applied here.

1. Batch reads and writes (never per-cell in a loop)

Reading 1,000 cells with `getValue()` in a loop = 1,000 service calls. Reading the same range once with `getValues()` = **1 call**. Same for `setValues()` vs `setValue()`.

- **Applied:** every list read uses one `getRange(...).getValues()` over the whole range (`valuesOf_`), not per-row reads. Bulk writes (`apiSaveServices` , the client-rename cascade `renameClientEverywhere_`) read a column once and write it back once with `setValues()`.
- **Rule of thumb:** if you're calling a Sheets method inside a `for` loop, stop and batch it.

2. Read each sheet once per request, then work in memory

Re-reading the same sheet multiple times in one operation is pure waste.

- **Applied to `apiListDocs`:** it *used* to count line items by calling `lineItemsFor_` once **per document**, and each of those re-read the entire Line Items sheet. With N invoices that was N full scans, $O(N \times \text{items})$, which is why it got slower as data grew. The count wasn't even displayed, so it's gone. Listing quotes/invoices is now a single sheet read.
- **Applied to the settings memo:** `getSetting_` reads the Settings sheet **once per execution** and serves every later lookup from memory (`settingsMap_`). One invoice PDF reads ~8 settings → was 8 sheet reads, now 1. `setSetting_` clears the memo so writes stay correct.

3. Bundle client → server round-trips

Each `google.script.run` call has real latency (auth + cold-start overhead). Fewer, fatter calls beat many small ones.

- **Applied to `apiBootstrap`:** one call returns settings, services, the dashboard, and all the status lists, so the app boots from a single round-trip.

- **Applied to** `apiListBilling` : the Billing screen returns estimates **and** invoices in one call (was two).

3b. Optimistic UI + background write queue (why it feels instant)

Server writes have latency, so instead of blocking on them the app applies your change to local state **immediately** and uploads the write in the **background**:

- You tap "Won" / reschedule / edit / create an estimate → the screen updates at once.
- The write goes into a **sequential FIFO queue** (so create-then-edit stays ordered) and uploads behind the scenes.
- A top-bar **sync badge** reports state: `🔄 Saving N...` → `✓ Saved` , or `⚠ N unsynced, retry` on failure. Failures auto-retry with backoff, then flag loudly and keep the change queued.
- Edits are idempotent and **persisted to** `localStorage` , so a queued write survives a reload and resumes on next open. A `beforeunload` warning fires if writes are still pending.
- **Creates** (new estimate) show a "⌚ Saving" placeholder instantly, then reconcile to the real server-assigned number when the write returns. Creates are session-only (replaying a create could duplicate a financial doc), covered by the unload warning.

This is safe here specifically because it's a small team that never edits the same record at once. No concurrent edits means no merge conflicts. **Send (email/PDF), Approve, and Delete stay synchronous by design**: you want explicit confirmation for money/email actions. (Add-contact stays synchronous too: its duplicate check is a required round-trip.)

4. Cache fetched lists in the app; only refetch after a change

The biggest everyday win. The app keeps each list in memory (`S.contacts` , `S.jobs` , `S.quotes` , `S.invoices` , `S.dashboard`). Switching tabs renders from that cache with **zero server calls**. Any action that writes to the sheet calls `invalidate()` , which clears the caches so the next view refetches fresh data. The `🔄 Refresh` button clears everything.

- **Net effect**: navigating around the app is instant; you only pay for a server call on the first view of a tab and right after you change something, which is unavoidable anyway.

5. Don't interleave reads and writes unnecessarily

Reading right after a write can force Apps Script to flush pending writes, adding latency.

- **Applied to** `apiSetJob` : after writing the new date it returns the value it just computed instead of reading the cell back.

6. Why we do not use server-side `CacheService` for the data

Apps Script offers `CacheService` (store JSON server-side for up to 6 hours). It's tempting, but this CRM's source of truth is the **Sheet**, which can change *outside* the app: you can edit a tab by hand, the mobile lead **Form** appends rows, and the daily **automations** write on a schedule. A server cache has no way to know those happened, so it would serve **stale data** with no reliable way to invalidate. The client-side cache (§4) is safe because it's per-session, is cleared on every in-app change, and you can always tap . If this ever becomes multi-user with heavy read load, revisit `CacheService` with an explicit version/invalidation key.

Archiving (built in): the main lever for staying fast at volume

Every read scans the live tabs, so the fewer rows they hold, the faster everything is. Use  **Settings** ▶ **Archive old paid & closed docs** (or the sheet menu  **CRM** ▶ **Invoices & estimates** ▶ **Archive paid & closed docs**) to move:

- **Accepted / Declined estimates** (a quote is either won → it has an invoice, or dead),
- **Paid invoices issued *before* the current month** (this month's stay so monthly revenue is exact), and
- **all their line items** (usually the biggest tab),

into  `Archived...` tabs. The live tabs shrink and reads speed up. Safeguards:

- **Lifetime revenue is preserved:** archived paid totals are carried in a document property and added back into the dashboard's Lifetime figure.
- **Jobs are left live**, so each client's **Total Spent** stays correct.
- **Idempotent:** re-running only archives newly-eligible rows; nothing is double-counted.
- The archive is done with batched reads/writes (one `getValues` + one `setValues` per tab), and it also compacts out any blank gaps left by deletes.

Run it whenever the app starts to feel heavy (e.g. monthly). Archived rows stay in the  `Archived...` tabs if you ever need to look one up.

Capacity note

The data tabs ship with 500 formatted rows. If a live tab ever approaches that after archiving, add rows in the sheet (Insert ▶ Rows); the app reads by actual content, so more rows just work.

Quick checklist if something feels slow

1. Is a Sheets call inside a loop? Batch it with `getValues` / `setValues`.
2. Is the same sheet read more than once per request? Read once, reuse.

3. Can two `google.script.run` calls become one server function? Bundle them.
 4. Did a new list view get added without caching? Give it an `S.*` cache + `invalidate()`.
 5. Redeploy a **new version** after code changes, or you're testing the old one.
-

Local-first rearchitecture (2026-07-17)

The web app is now **local-first**. `doGet()` inlines the ENTIRE dataset into the page (via an `HtmlService` template + a `<script type="application/json" id="__BOOT__">` block), so the app opens with **zero** `google.script.run` **round trips**. After boot, the browser owns the live state: every read (navigating, opening a client/invoice, its line items, the price book, the dashboard) is served from memory, no server call. The dashboard is recomputed client-side (`computeDashboard()`), never fetched.

Why: measured on-device latency was ~3.9s **per** `google.script.run` call, independent of data size, so the win is eliminating the calls, not speeding them up.

Server JSON hot store. The boot payload is persisted to a Drive file `crm-data.json` (chosen over `CacheService` / `PropertiesService` because 3,000 records exceed the 500 KB Properties cap). `doGet` serves that file directly; it is rebuilt from Sheets only when a change the app did *not* make marks it stale. Google Sheets remains the durable source of truth; any drift self-heals on the next rebuild.

Reverse sync (Sheets → JSON): the deliberate exception to the "no server cache" rule above. A change made outside the app sets a stale flag, so the next open rebuilds the JSON from Sheets:

- a manual sheet edit (installable `onEdit`, which does NOT fire on the app's own `setValues`);
- a mobile lead-form submission (`onFormSubmit`);
- the daily automations (`markOverdueInvoices`, `rollForwardRecurringJobs`, `remindUpcomingJobs`, `sendReviewRequests`) call `DATA_markStale()` when they change data. Single-writer is what makes serving a cached snapshot safe: there are no concurrent editors to go stale against within a session, and ⌂ / reload force a fresh rebuild on demand.

Writes stay optimistic. Edits apply locally instantly and upload in the background (FIFO queue, retry, durable across reload). A newly created record carries a temp id until the server assigns the real one; `remapPending()` rewrites any queued dependent write (estimate→client, invoice→estimate) to the real id the moment it arrives; this is the fix for the "pick a valid client" race where creating an estimate right after a client used to fail.

Client-assigned sequential IDs (2026-07-17, build g)

Because there is one writer, the app assigns the next real `CL-` / `EST-` / `INV-` number itself (seeded from the `_meta` counters carried in the boot payload), so a new record shows its **final** number instantly, with no "temp16" placeholder, and the number never changes out from under an open screen (which previously broke Edit / Approve / "pick a valid client" when the server renumbered mid-action). The server (`db.gs` `insertMany` / `DB_useProvidedId`) adopts a provided id if it's free (advancing the

counter) and only hands back a corrected id on the rare collision with a lead-form/automation row; the client absorbs that via `remapPending` + `advanceCounter`. Line-item and job ids stay server-assigned (never user-visible). Archiving is now optimistic (queued, rolled back on server rejection) so it is never blocked by a background save. Mutations re-render the *current* screen via `rerender()`, so acting on a Home week-job / follow-up updates Home (and its locally-recomputed dashboard) immediately.